

UNIDAD V: Colas con Prioridad y Montones (Heaps)

1. Introducción - Breve Historia

Las colas con prioridad y los montones son estructuras de datos fundamentales en la ciencia de la computación que han evolucionado a lo largo de décadas para resolver problemas donde el orden de procesamiento es crucial.

La idea de una cola con prioridad surgió en los primeros días de la informática, en la década de 1960, cuando los sistemas operativos necesitaban gestionar procesos con diferentes niveles de importancia. Los primeros sistemas utilizaban implementaciones simples basadas en listas ordenadas.

El montón binario (binary heap), la implementación más común de una cola con prioridad, fue introducido formalmente por J.W.J. Williams en 1964 como parte del algoritmo Heapsort. Este avance permitió operaciones más eficientes y fue fundamental para el desarrollo de algoritmos posteriores como el algoritmo de Dijkstra para encontrar caminos más cortos en grafos (1959), que utiliza colas de prioridad para su implementación eficiente.

A partir de los años 70, surgieron variantes más sofisticadas como los montones de Fibonacci, desarrollados por Michael L. Fredman y Robert E. Tarjan en 1984, que mejoraron la eficiencia teórica de ciertas operaciones.

Hoy en día, las colas con prioridad y los montones son componentes esenciales en áreas como:

- Sistemas operativos (planificación de procesos)
- Algoritmos de grafos
- Compresión de datos (códigos de Huffman)
- Simulaciones de eventos discretos
- Algoritmos de búsqueda

2. Definiciones y Propósito

Colas con Prioridad

Definición: Una cola con prioridad es una estructura de datos abstracta similar a una cola regular, pero donde cada elemento tiene asignada una "prioridad". Los elementos con mayor prioridad son atendidos antes que los elementos con menor prioridad, independientemente del orden de llegada.

Propósito:

- Gestionar recursos limitados cuando las solicitudes tienen diferentes niveles de urgencia o importancia
- Planificar tareas según su prioridad
- Implementar algoritmos como Dijkstra, Prim, Huffman, etc.
- Simular sistemas donde ciertos eventos deben procesarse antes que otros

Operaciones principales:

- `insert(elemento, prioridad)`: Añade un elemento con una prioridad determinada
- `extract_max()` o `extract_min()`: Elimina y devuelve el elemento con la máxima (o mínima) prioridad
- `peek()`: Consulta el elemento con la máxima (o mínima) prioridad sin eliminarlo

Montones (Heaps)

Definición: Un montón es una estructura de datos basada en un árbol que satisface la propiedad de montón: para cada nodo, el valor del nodo es mayor (o menor) que los valores de sus hijos. Existen dos tipos principales:

- Montón máximo (Max Heap): El valor de cada nodo es mayor o igual que el de sus hijos

- Montón mínimo (Min Heap): El valor de cada nodo es menor o igual que el de sus hijos

Propósito:

- Implementar colas con prioridad de manera eficiente
- Realizar ordenamientos (Heapsort)
- Encontrar los k elementos más grandes/pequeños en un conjunto
- Gestionar recursos en sistemas operativos

Propiedades clave:

- Estructura parcialmente ordenada (no totalmente)
- Completo o casi completo como árbol
- Puede representarse eficientemente como un arreglo
- Ofrece operaciones de inserción y extracción en tiempo logarítmico $O(\log n)$

3. Implementación Paso a Paso con Código en Python

3.1 Implementación de una Cola con Prioridad usando `heapq`

Python proporciona el módulo `heapq` que implementa un montón mínimo:

```
import heapq

# Crear una cola con prioridad vacía
cola_prioridad = []

# Insertar elementos (elemento, prioridad)
# Nota: heapq implementa un min-heap, así que valores más pequeños = mayor prioridad
heapq.heappush(cola_prioridad, (5, "Tarea con prioridad media"))
heapq.heappush(cola_prioridad, (1, "Tarea urgente"))
heapq.heappush(cola_prioridad, (10, "Tarea de baja prioridad"))

# Extraer el elemento con mayor prioridad (menor valor)
prioridad, tarea = heapq.heappop(cola_prioridad)
print(f"Tarea a realizar: {tarea} (Prioridad: {prioridad})")

# Consultar el siguiente elemento sin extraerlo
if cola_prioridad:
    print(f"Siguiente tarea: {cola_prioridad[0][1]} (Prioridad: {cola_prioridad[0][0]})")
```

3.2 Implementación Personalizada de un Montón Binario (Min Heap)

```
class MinHeap:
    def __init__(self):
        self.heap = []
        self.size = 0

    def parent(self, i):
        return (i - 1) // 2

    def left_child(self, i):
        return 2 * i + 1

    def right_child(self, i):
```

```

        return 2 * i + 2

def get_min(self):
    if self.size > 0:
        return self.heap[0]
    return None

def insert(self, key):
    self.heap.append(key)
    self.size += 1
    self._sift_up(self.size - 1)

def extract_min(self):
    if self.size == 0:
        return None

    min_item = self.heap[0]

    # Reemplazar la raíz con el último elemento
    self.heap[0] = self.heap[self.size - 1]
    self.size -= 1
    self.heap.pop()

    # Restaurar la propiedad de montón
    if self.size > 0:
        self._sift_down(0)

    return min_item

def _sift_up(self, i):
    parent_idx = self.parent(i)

    # Si el elemento actual es menor que su padre, intercambiarlos
    if i > 0 and self.heap[i] < self.heap[parent_idx]:
        self.heap[i], self.heap[parent_idx] = self.heap[parent_idx], self.heap[i]
        self._sift_up(parent_idx)

def _sift_down(self, i):
    min_idx = i
    left = self.left_child(i)
    right = self.right_child(i)

    # Encontrar el índice del elemento más pequeño entre el actual, izquierdo y derecho
    if left < self.size and self.heap[left] < self.heap[min_idx]:
        min_idx = left

    if right < self.size and self.heap[right] < self.heap[min_idx]:
        min_idx = right

    # Si el elemento más pequeño no es el actual, intercambiarlos y continuar
    if min_idx != i:
        self.heap[i], self.heap[min_idx] = self.heap[min_idx], self.heap[i]
        self._sift_down(min_idx)

```

Ejemplo de uso

```

heap = MinHeap()
heap.insert(5)
heap.insert(3)
heap.insert(8)
heap.insert(1)
heap.insert(10)

print("Min element:", heap.get_min()) # 1

# Extraer elementos en orden de prioridad
while heap.size > 0:
    print(heap.extract_min(), end=" ") # 1 3 5 8 10

```

3.3 Implementación de una Cola con Prioridad Personalizada

```

class PriorityQueue:
    def __init__(self):
        self.queue = []
        self.size = 0

    def is_empty(self):
        return self.size == 0

    def insert(self, item, priority):
        # Encontrar la posición correcta para el nuevo elemento
        i = 0
        while i < self.size and priority >= self.queue[i][0]:
            i += 1

        # Insertar el elemento en la posición correcta
        self.queue.insert(i, (priority, item))
        self.size += 1

    def get_highest_priority(self):
        if not self.is_empty():
            return self.queue[0][1]
        return None

    def remove_highest_priority(self):
        if self.is_empty():
            return None

        self.size -= 1
        return self.queue.pop(0)[1]

# Ejemplo de uso
pq = PriorityQueue()
pq.insert("Tarea A", 2)
pq.insert("Tarea Urgente", 1)
pq.insert("Tarea Menos Importante", 3)

print("Tarea con mayor prioridad:", pq.get_highest_priority())

# Procesar tareas en orden de prioridad

```

```
while not pq.is_empty():
    print("Procesando:", pq.remove_highest_priority())
```

4. Caso de la Vida Real: Sistema de Gestión de Emergencias Hospitalarias

Problema:

Un hospital necesita un sistema para gestionar la atención de pacientes en la sala de emergencias. Los pacientes deben ser atendidos según la gravedad de su condición, no por orden de llegada.

Solución con Colas de Prioridad:

Paso 1: Definir el modelo de paciente y las prioridades

```
class Paciente:
    def __init__(self, nombre, edad, sintomas, hora_llegada):
        self.nombre = nombre
        self.edad = edad
        self.sintomas = sintomas
        self.hora_llegada = hora_llegada
        self.prioridad = None # Se asignará tras el triaje

    def __str__(self):
        return f"{self.nombre} ({self.edad} años) - Prioridad: {self.prioridad}"

# Sistema de triaje: niveles de gravedad
# 1: Resucitación (atención inmediata)
# 2: Emergencia (muy urgente, minutos)
# 3: Urgente (hasta 1 hora)
# 4: Menos urgente (hasta 2 horas)
# 5: No urgente (hasta 4 horas)
```

Paso 2: Implementar la cola de prioridad para la sala de emergencias

```
import heapq
from datetime import datetime, timedelta

class SalaEmergencias:
    def __init__(self):
        self.cola_pacientes = [] # Min-heap
        self.contador = 0 # Para desempatar pacientes con la misma prioridad

    def asignar_prioridad(self, paciente, nivel_triage):
        """Asigna prioridad según nivel de triaje (1-5) y registra al paciente."""
        paciente.prioridad = nivel_triage

        # Usamos una tupla: (nivel_triage, contador, paciente)
        # El contador asegura que pacientes con la misma prioridad
        # sean atendidos por orden de llegada (FIFO)
        heapq.heappush(self.cola_pacientes,
                       (nivel_triage, self.contador, paciente))
        self.contador += 1

    def siguiente_paciente(self):
```

```

    """Retorna el siguiente paciente a atender."""
    if not self cola_pacientes:
        return None

    _, _, paciente = heapq.heappop(self cola_pacientes)
    return paciente

def pacientes_en_espera(self):
    """Retorna el número de pacientes en espera."""
    return len(self cola_pacientes)

def ver_siguiente(self):
    """Consulta el siguiente paciente sin eliminarlo."""
    if not self cola_pacientes:
        return None

    return self cola_pacientes[0][2]

```

Paso 3: Uso del sistema en un escenario real

```

# Simulación de un día en la sala de emergencias
def simular_sala_emergencias():
    sala = SalaEmergencias()
    hora_actual = datetime(2023, 4, 15, 8, 0) # 8:00 AM

    # Llegada de pacientes
    pacientes = [
        Paciente("Carlos Rodríguez", 78, ["dolor en el pecho", "dificultad para respirar"],
            hora_actual + timedelta(minutes=10)),
        Paciente("María González", 45, ["dolor abdominal intenso"],
            hora_actual + timedelta(minutes=15)),
        Paciente("Pedro Sánchez", 25, ["torcedura de tobillo"],
            hora_actual + timedelta(minutes=20)),
        Paciente("Ana Martínez", 30, ["fiebre alta", "dolor de cabeza"],
            hora_actual + timedelta(minutes=25)),
        Paciente("Luis Hernández", 60, ["mareo", "presión arterial alta"],
            hora_actual + timedelta(minutes=30)),
        Paciente("Elena Pérez", 8, ["fractura expuesta de brazo"],
            hora_actual + timedelta(minutes=35)),
        Paciente("Juan Díaz", 40, ["corte profundo en mano"],
            hora_actual + timedelta(minutes=40))
    ]

    # Triage: evaluación y asignación de prioridades
    print("=== PROCESO DE TRIAJE ===")
    # Carlos: posible infarto, prioridad máxima
    sala.asignar_prioridad(pacientes[0], 1)
    print(f"Paciente registrado: {pacientes[0]} a las {pacientes[0].hora_llegada.strftime('%H:%M')}")

    # María: dolor agudo pero estable
    sala.asignar_prioridad(pacientes[1], 3)
    print(f"Paciente registrado: {pacientes[1]} a las {pacientes[1].hora_llegada.strftime('%H:%M')}")

    # Pedro: caso no urgente

```

```

sala.asignar_prioridad(pacientes[2], 5)
print(f"Paciente registrado: {pacientes[2]} a las {pacientes[2].hora_llegada.strftime('%H:%M')}")

# Ana: caso que requiere atención
sala.asignar_prioridad(pacientes[3], 4)
print(f"Paciente registrado: {pacientes[3]} a las {pacientes[3].hora_llegada.strftime('%H:%M')}")

# Luis: requiere monitoreo
sala.asignar_prioridad(pacientes[4], 3)
print(f"Paciente registrado: {pacientes[4]} a las {pacientes[4].hora_llegada.strftime('%H:%M')}")

# Elena: trauma grave, alta prioridad
sala.asignar_prioridad(pacientes[5], 2)
print(f"Paciente registrado: {pacientes[5]} a las {pacientes[5].hora_llegada.strftime('%H:%M')}")

# Juan: requiere sutura
sala.asignar_prioridad(pacientes[6], 3)
print(f"Paciente registrado: {pacientes[6]} a las {pacientes[6].hora_llegada.strftime('%H:%M')}")

# Atención de pacientes según prioridad
print("\n=== ATENCIÓN DE PACIENTES ===")
hora_actual = hora_actual + timedelta(minutes=45) # 8:45 AM

while sala.pacientes_en_espera() > 0:
    paciente = sala.siguiente_paciente()
    tiempo_espera = (hora_actual - paciente.hora_llegada).total_seconds() / 60

    print(f"Atendiendo a {paciente.nombre} a las {hora_actual.strftime('%H:%M')}")
    print(f" Nivel de prioridad: {paciente.prioridad}")
    print(f" Tiempo de espera: {int(tiempo_espera)} minutos")

    # Simulamos el tiempo de atención según la prioridad
    if paciente.prioridad == 1:
        hora_actual += timedelta(minutes=30) # Casos graves requieren más tiempo
    elif paciente.prioridad == 2:
        hora_actual += timedelta(minutes=25)
    else:
        hora_actual += timedelta(minutes=20)

    print(f" Hora de finalización: {hora_actual.strftime('%H:%M')}")
    print()

# Ejecutar simulación
simular_sala_emergencias()

```

Explicación del Caso de Uso:

1. Modelo del sistema:

- Cada paciente tiene información personal y un nivel de prioridad asignado (1-5).
- La sala de emergencias utiliza una cola de prioridad para gestionar a los pacientes.
- Los pacientes con condiciones más graves (prioridad 1) son atendidos primero.

2. Mecanismo de prioridad:

- Se utiliza un heap mínimo donde menor número = mayor prioridad (1 es lo más urgente).

- Se incluye un contador como segundo elemento de la tupla para desempatar pacientes con la misma prioridad, asegurando que se respete el orden de llegada dentro del mismo nivel de urgencia.

3. Funcionamiento en la simulación:

- Los pacientes llegan en distintos momentos y se les asigna un nivel de triaje.
- El sistema automáticamente ordena a los pacientes según su gravedad.
- La atención se proporciona en orden de prioridad, no por orden de llegada.
- Se calcula y muestra el tiempo de espera para cada paciente.

4. Beneficios del uso de colas con prioridad:

- Gestión eficiente de recursos médicos limitados
- Atención prioritaria a casos que pueden ser críticos
- Balance entre urgencia médica y tiempo de llegada
- Capacidad para reorganizar la cola cuando llegan casos más urgentes

Esta implementación refleja cómo funcionan los sistemas reales de triaje hospitalario, donde la prioridad médica prevalece sobre el orden de llegada, pero se mantiene un orden FIFO dentro del mismo nivel de urgencia.

5. Consideraciones Adicionales

Complejidad Temporal de las Operaciones

Operación	Cola con Prioridad basada en Heap	Cola con Prioridad basada en Lista
Inserción	$O(\log n)$	$O(n)$
Extracción del máximo/mínimo	$O(\log n)$	$O(1)$
Consulta del máximo/mínimo	$O(1)$	$O(1)$

Variantes Avanzadas de Heaps

- **Fibonacci Heap:** Mejora las operaciones de decremento de clave y unión
- **Binomial Heap:** Eficiente para operaciones de unión de montones
- **Leftist Heap:** Especializado para operaciones de fusión
- **Skew Heap:** Versión auto-ajutable del leftist heap

Aplicaciones Adicionales

- **Algoritmos de grafos:** Dijkstra, Prim
- **Ordenamiento externo:** Cuando los datos no caben en memoria
- **Programación de tareas en sistemas operativos**
- **Compresión de datos (códigos de Huffman)**
- **Selección de k elementos más grandes/pequeños**

Las colas con prioridad y los montones son estructuras versátiles y eficientes que permiten resolver una amplia gama de problemas computacionales donde la prioridad o el orden son factores críticos.

